

Frontend, backend ja andmebaas

11IT · Veebirakendused · viies tund · konspekt

Selle tunni suur mõte: peaaegu iga rakendus, mida kasutad, on kolmest tükist. Üks näitab (frontend), teine otsustab (backend) ja kolmas mäletab (andmebaas). Omavahel suhtlevad nad päringute ja vastustega.

Mis juhtub, kui vajutad „Meeldib“?

Vajutad Instagramis südamele. Süda läheb kohe punaseks. Sinu sõber teises linnas avab sama pildi ja näeb, et meeldimisi on ühe võrra rohkem. Kuidas tema telefon sellest teada sai?

Sinu telefon ei saatnud midagi otse sõbra telefoni. Ta saatis teate Instagrami serverile, server kirjutas selle andmebaasi ja kui sõber pildi avas, küsis tema telefon serverilt värsked andmed. Kõik, mida selles kursuses ehitame, käib täpselt sama skeemi järgi.

Restoran, aga nüüd täielik

API konspektis võrdlesime serverit köögiga. Nüüd vaatame kogu restorani:

Restoranis	Rakenduses	Mida teeb
Saal, menüü ja laud	Frontend	Kõik, mida kasutaja näeb ja millele klõpsab. Töötab sinu brauseris.
Kelner ja tellimuse lipik	HTTP päring ja API	Viib soovi saalist kööki ja vastuse tagasi. Kokkulepitud kujul.
Köök ja kokk	Backend	Võtab tellimuse vastu, kontrollib, teeb töö ära. Klient kööki ei näe.
Külmkamber ja ladu	Andmebaas	Hoiab kõike, mida on vaja homme ka mäletada.

Pane tähele: klient ei lähe kunagi ise külmkappi. Ta räägib ainult kelneriga. Frontend ei räägi kunagi otse andmebaasiga, alati läbi backendi.

Frontend: see, mida näed

Frontend on kõik, mis jõuab sinu brauserisse: HTML, CSS ja JavaScript. Meie kirjutame seda Reactiga ja TypeScriptiga. Frontendi töö on näidata andmeid ilusti ja reageerida kohe, kui kasutaja midagi teeb.

Üks asi, mis paljusid üllatab: **kogu frontendi kood on kasutaja käes**. Vajuta mõnel lehel F12 ja näed seda. Kasutaja võib seda koodi oma brauseris ka muuta. Proovi: paremklõpsa e-poes hinnal, vali „Inspect“ ja kirjuta 999 € asemele 1 €. Ekraanil ongi hind 1 €. Aga ostukorvi jõudes on see jälle 999 €, sest päris hind on poe serveris.

Backend: see, mis otsustab

Backend on programm, mis töötab serveris ehk arvutis kuskil andmekeskuses. Kasutaja seda ei näe ega saa muuta. Tal on neli põhitööd:

- võtab päringu vastu (näiteks „lisa see arvustus“);
- kontrollib, kes sa oled ja kas sul on lubatud seda teha;
- kontrollib reegleid: kas hinne on 1 kuni 5, kas toote hind on õige, kas laos on kaupa;
- räägib andmebaasiga ja saadab vastuse tagasi.

Backendi võib kirjutada mitmes keeles: Python, Java, C#, Go. Üks levinumaid on **Node.js**, mis käivitab JavaScripti serveris. See tähendab, et backendi jaoks ei pea uut keelt õppima.

Andmebaas: see, mis mäletab

Andmebaas hoiab andmeid nii, et need jäävad alles ka siis, kui server taaskäivitub, ja et neist saab kiiresti otsida. Kõige levinum on **tabelitega** andmebaas. See on nagu Exceli tabel: igal real on üks asi (üks arvustus) ja igas veerus üks omadus.

id	toit	hinne	kommentaar	lisatud
1	Kanakaste riisiga	5	Parim toit sel nädalal!	2026-10-06 11:42
2	Hernesupp	3	Natuke liiga paks.	2026-10-06 11:45
3	Makaronid hakklihaga	4	Klassika, alati hea.	2026-10-06 12:03

Andmebaasiga räägitakse keeles **SQL**. Seda õpime hiljem, aga juba praegu on näha, et see on peaaegu inglise keel:

```
SELECT * FROM arvustused WHERE hinne >= 4;

INSERT INTO arvustused (toit, hinne, kommentaar)
VALUES ('Kalapulgad', 2, 'Kuivad.');
```

Esimene ütleb: „näita kõiki arvustusi, mille hinne on vähemalt 4“. Teine lisab uue rea. Uue rea *id* ja lisamise aja paneb andmebaas ise.

Kuidas nad omavahel räägivad

Frontend ja backend räägivad **HTTP** abil, täpselt nagu fetch API tunnis. Igal päringul on **meetod** (mida teha) ja **aadress** (millega). Vajadusel on kaasas ka sisu, tavaliselt JSON-ina. Meie söökla rakenduse API võiks olla selline:

Meetod ja aadress	Mida teeb	Vastus
GET /api/arvustused	Annab kõik arvustused	200 ja nimekiri
GET /api/arvustused/7	Annab ühe arvustuse, mille id on 7	200 või 404
POST /api/arvustused	Lisab uue arvustuse, sisu on kaasas	201 ja uus arvustus
DELETE /api/arvustused/7	Kustutab arvustuse, mille id on 7	204 (vastus on tühi)

Kui uus arvustus saadetakse backendile, näeb päringu sisu välja nii:

```
{ "toit": "Kalapulgad", "hinne": 2, "kommentaar": "Kuivad." }
```

Backend vastab sama asjaga, aga lisab juurde id ja aja, mille andmebaas andis:

```
{ "id": 4, "toit": "Kalapulgad", "hinne": 2, "kommentaar": "Kuivad.",
  "lisatud": "2026-10-06T12:10:00Z" }
```

Uued staatusekoodid lisaks 200-le, 404-le ja 500-le: **201** tähendab „loodud“, **400** „sinu päring on vigane“ (näiteks hinne on 7) ja **401** „logi enne sisse“.

Ühe klõpsu teekond

Paneme kõik kokku. Mari kirjutab arvustuse ja vajutab „Lisa“:



1. React saadab päringu: POST /api/arvustused ja kaasas JSON uue arvustusega.
2. Backend kontrollib: kas oled sisse logitud? Kas hinne on 1 kuni 5?
3. Backend käsib andmebaasil arvustuse salvestada.
4. Andmebaas salvestab ja annab uuele reale numbri (id).
5. Backend vastab: staatusekood 201 ja JSON salvestatud arvustusega.
6. React lisab arvustuse nimekirja ja sina näed seda ekraanil.

Kogu ring võtab tavaliselt alla poole sekundi. Sammud 2–4 toimuvad serveris ja kasutaja neid ei näe.

Miks mitte kõike frontendis teha?

Tänases tunnis teemegi kõik frontendis ja see töötab. Siis vajutad F5 ja kõik su arvustused on läinud. Sellel on kolm põhjust, miks päris rakendus vajab backendi:

Probleem	Mis juhtub ilma backendita
Mälu	Andmed on ainult sinu brauseri mälus. Lehe värskendamisel kaovad ja pinginaaber neid kunagi ei näe.
Usaldus	Igaüks saab F12-ga frontendi koodi muuta ja saata hinde 1000. Frontendi kontroll on kasutaja mugavuseks, backendi kontroll on turvalisuseks.
Saladused	Paroole, makseandmeid ja API võtmeid ei saa frontendis panna, sest kogu frontendi kood on kõigile nähtav.

Reegel, mis kehtib igas firmas: ära kunagi usalda seda, mis tuleb frontendist. Kõik olulised kontrollid tehakse uuesti backendis.

Kus on TypeScript selles pildis

Frontend ja backend peavad olema kokku leppinud, mis kujul andmed liiguvad. Kui backend saadab välja nimega food ja frontend ootab välja toit, ei tööta midagi ja ekraanil on tühjus. TypeScriptis kirjutame selle kokkuleppe üles tüübina:

```
type Arvustus = {
  id: number;
  toit: string;
  hinne: number;
  kommentaar: string;
};
```

Nüüd kontrollib VS Code iga koha üle, kus arvustust kasutad. Kui kirjutad arvustus.food, joonib ta selle punaseks enne, kui oled lehte avanudki.

Kuhu me liigume

Millal	Mida teeme
Täna	Frontend: söökla arvustuste rakendus Reactis. Andmed on failis, mis teeb näo, et on backend.
Järgmistes tundides	Backend: väike server, mis oskab arvustusi anda, lisada ja kustutada.
Seejärel	Andmebaas: arvustused jäävad alles ka pärast serveri taaskäivitust.
Lõpuks	Ühendame kõik kokku: React küsib andmed fetch-iga sinu enda backendist.

Mõisted

Mõiste	Lühidalt
frontend	rakenduse osa, mis töötab kasutaja brauseris
backend	programm serveris, mis võtab päringuid vastu, kontrollib ja vastab
andmebaas	koht, kus andmed püsivad ja kust neid saab kiiresti otsida
server	arvuti, kus backend töötab
API	kokkulepe, milliseid päringuid backend mõistab ja mida ta vastab
HTTP meetod	GET (anna), POST (lisa), PUT/PATCH (muuda), DELETE (kustuta)
JSON	tekstikuju, milles andmed frontendi ja backendi vahel liiguvad
SQL	keel, millega andmebaasilt küsitakse ja sinna kirjutatakse
tüüp (type)	TypeScriptis kirja pandud kokkulepe, mis kujul andmed on